UI5 Practice Lab

Five small projects: freestyle CRUD, Fiori app activation, extension, deployment and the UI5 Showroom

Edition: TASKS ONLY (no code, no answers)

Made for fast practice

- You already know the concepts, so every story is a short **do it** list with acceptance criteria. No theory chapters.
- Each project is a single small deliverable. Bracketed topics of your list (navigation, custom control, filter, group, currency, time offset, CSS, authorization simulation) live together in **P5 Showroom**.
- Hints name topics, not solutions. A second edition with answers exists.
- S/4 tasks (P2, P3, P4.3) need a system; transaction names can differ by release, so verify them in your system.

Projects

#	Project	Deliverable	Effort
P0	Shared Setup: Mock Backend and UI5 Tooling	A tiny CAP backend (Customers, Orders) serving OData V4 and V2, plus a working UI5 development setup. All later projects use it.	2-3 hours
P1	Freestyle CRUD App (V4 and V2)	Order Manager: one table with create, edit and delete, built once with the OData V4 model and once with V2.	6-8 hours
P2	Standard Fiori App Activation (S/4HANA lab)	Activate one SAP-delivered Fiori app end to end on an S/4 system, make it appear on the launchpad for a test user and learn the cache and debug toolbox.	4-6 hours (needs a system)
P3	Extend a Standard List Report	Extend one standard (or self-built) Fiori elements list report without touching the original code: hide a field, add a column, add a filter, add a button with logic.	6-8 hours
P4	Deploy a UI5 App to BTP and to S/4HANA	Deploy the freestyle CRUD app from P1 to SAP BTP (Cloud Foundry) and to an S/4 ABAP repository and open it from a launchpad in both worlds.	6-8 hours
P5	UI5 Showroom: Navigation, Control, Filter, Group, Currency, Time Offset, CSS, Authorization	One small app "Order Showroom" (two tiny apps for navigation) that demonstrates every item of your list on the mock backend.	10-14 hours

Suggested pace (about 3 weeks of the 2-month plan)

Week	Projects	Focus
1	P0, P1	Backend, tooling, freestyle CRUD (V4 and V2)
2	P5	Showroom: navigation, control, filter/group, currency, time zone, CSS, authorization
3	P2, P3, P4	S/4 lab: activation, extension, deployment (needs system access; do P4.2 on BTP trial anytime)

Learning links

UI5 documentation and learning

- **SAPUI5 / OpenUI5 SDK: documentation, API reference, samples, Walkthrough tutorial**: https://ui5.sap.com/
- **UI5 Tooling (CLI, build, serve, middleware)**: https://sap.github.io/ui5-tooling/
- **easy-ui5 generator**: https://github.com/SAP/easy-ui5

Fiori and S/4HANA

- **SAP Fiori Apps Reference Library (implementation information for standard apps)**: https://fioriappslibrary.hana.ondemand.com/
- **SAP Fiori tools documentation (generator, adaptation project, deployment)**: https://help.sap.com/docs/SAP_FIORI_tools
- **SAP Fiori design guidelines**: https://experience.sap.com/fiori-design-web/
- **SAP Community: Fiori and SAPUI5 topics**: https://community.sap.com/topics/fiori-elements

Backend and samples

- **CAP documentation**: https://cap.cloud.sap/docs/
- **OData V2 adapter for CAP**: https://github.com/cap-js-community/odata-v2-adapter
- **Public Northwind OData V2 service**: https://services.odata.org/V2/Northwind/Northwind.svc/
- **Public Northwind OData V4 service**: https://services.odata.org/V4/Northwind/Northwind.svc/
- **SAP-samples on GitHub (UI5, Fiori elements, CAP examples)**: https://github.com/SAP-samples

Project P0 - Shared Setup: Mock Backend and UI5 Tooling

Item	Detail
Mini project	A tiny CAP backend (Customers, Orders) serving OData V4 and V2, plus a working UI5 development setup. All later projects use it.
Level / effort	Easy / 2-3 hours
Topics practised	CAP service V4 + V2 adapter, UI5 CLI / Fiori tools, local preview.

Learning outcomes
- One backend for every project below.
- A working command to start a UI5 app with a proxy to the backend.

Story P0.1 - Backend with V4 and V2

Goal: A service you can call from both model types.

Your tasks

1. Create a CAP project shop-backend with Customers (name, country, email) and Orders (order number, customer, category, amount, currency, status, orderedAt timestamp, deliveredAt timestamp).
2. Add 5 customers and 20 orders by CSV (mixed currencies EUR/USD/INR, 4 categories, mixed status, timestamps in UTC).
3. Expose the service on path shop (V4). Add an OData V2 endpoint for the same service using the community V2 adapter.
4. Add a function me() that returns the logged-in user id and roles (used in P5).
5. Test V4 and V2 URLs in the browser, check $metadata of both.

Where to look / topics to explore
- CAP + @cap-js-community/odata-v2-adapter
- Alternative with no backend: public Northwind V2 and V4 services

Acceptance criteria
- Both /odata/v4/shop and the V2 URL return orders.

Story P0.2 - UI5 tooling

Goal: Create, run and debug a UI5 app locally.

Your tasks

1. Install the UI5 CLI and a generator of your choice (Fiori tools application generator or easy-ui5).
2. Generate a freestyle app named shop.crud with a proxy from /odata to the CAP server (CORS-free).
3. Run it, open browser dev tools, set a breakpoint in the controller.

Where to look / topics to explore
- ui5.yaml: fiori-tools-proxy or ui5-middleware-simpleproxy

Acceptance criteria
- The app starts and calls the backend without CORS errors.

Milestone checkpoint

Self-test questions (write your own answers)

1. Why use a proxy in ui5.yaml?

Project P1 - Freestyle CRUD App (V4 and V2)

Item	Detail
Mini project	Order Manager: one table with create, edit and delete, built once with the OData V4 model and once with V2.
Level / effort	Intermediate / 6-8 hours
Topics practised	Manifest models, ODataModel V4 (bindList create, context delete, auto groups), ODataModel V2 (create, update, remove, submitChanges), messages, busy handling.

Learning outcomes
- Implement CRUD with both models and know the differences.
- Handle errors and busy state.

Story P1.1 - V4 list and edit

Goal: Read and inline-edit orders.

Your tasks

1. Configure a V4 model in manifest.json for the shop service.
2. Show orders in a growing responsive table (order number, category, amount, currency).
3. Make cells editable; changes must be saved to the backend automatically. Verify with the network tab which request is sent.

Where to look / topics to explore
- sap.ui.model.odata.v4.ODataModel: autoExpandSelect, groupId $auto

Acceptance criteria
- Editing a cell sends a PATCH.

Story P1.2 - V4 create and delete

Goal: Add and remove rows.

Your tasks

1. Button New adds a row (use the list binding create) and the row is saved when complete.
2. Button Delete removes the selected row after a confirmation dialog.
3. Show success and error messages; test by creating an order with missing mandatory data (add a validation on the backend).

Where to look / topics to explore
- ODataListBinding.create, Context.delete, created() promise, MessageBox.confirm

Acceptance criteria
- Create, edit and delete all work; a backend error shows a readable message.

Story P1.3 - V2 version of the same app

Goal: Same features with the V2 model.

Your tasks

1. Generate (or copy) a second app shop.crudv2 using the V2 endpoint.
2. Implement create, update and delete with the model methods and callbacks. Make editing deferred: nothing is sent until the user presses Save; add a Cancel (reset changes).

3. Compare batch behaviour in the network tab.

Where to look / topics to explore

- ODataModel V2: create, update, remove, submitChanges, resetChanges, setDeferredGroups, hasPendingChanges

Acceptance criteria

- Edits are sent only after Save; Cancel restores the old values.

Story P1.4 - Comparison sheet

Goal: Know V2 versus V4 for interviews and projects.

Your tasks

1. Fill a table: model creation, binding syntax differences, change handling, batch and groups, messages, side effects, expand/select, metadata caching, when to choose which.

Acceptance criteria

- Your table has at least 8 rows.

Milestone checkpoint

Self-test questions (write your own answers)

1. How are changes submitted in V4 by default?
2. How do you cancel pending edits in V2?

Project P2 - Standard Fiori App Activation (S/4HANA lab)

Item	Detail
Mini project	Activate one SAP-delivered Fiori app end to end on an S/4 system, make it appear on the launchpad for a test user and learn the cache and debug toolbox.
Level / effort	Intermediate / 4-6 hours (needs a system)
Topics practised	Fiori Apps Reference Library, OData service activation, ICF nodes, launchpad catalogs and roles, cache invalidation, error logs, UI5 debugging.

Learning outcomes

- Activate any standard app systematically.
- Clear the right caches in the right order.
- Find errors fast using logs and browser tools.

Story P2.1 - Analyse the app

Goal: Know what must be activated.

Your tasks

1. Pick one standard app (for example a list report such as a Manage/Display app) in the Fiori Apps Reference Library.
2. From the Implementation Information note: OData service name, UI5 component/app name, catalog and group/role names, required back-end authorisation objects.
3. Write them in a one-page activation sheet.

Where to look / topics to explore

- Fiori Apps Reference Library: Implementation Information tab

Acceptance criteria

- Your activation sheet lists service, ICF node, catalog, role.

Story P2.2 - Activate service and UI5 app

Goal: Back end and front-end ready.

Your tasks

1. Activate the OData service in the gateway (system alias, package, ICF node).
2. Check the UI5 app ICF node is active.
3. Test the OData service directly with the gateway client and the metadata URL.

Where to look / topics to explore

- Gateway service maintenance, SICF, gateway client

Acceptance criteria

- $metadata and a read request return data in the gateway client.

Story P2.3 - Launchpad: catalog, role, tile

Goal: The tile shows for a test user.

Your tasks

1. Find the standard business catalog of the app and assign it to a role; assign the role to a test user.
2. Alternative: create a custom catalog, add the app via target mapping and tile, and put it in a group/space/page.
3. Log in as the test user and open the app; check the semantic object and action of the target mapping.

Where to look / topics to explore

- Launchpad designer (client-specific and cross-client), PFCG roles, intent = semantic object + action

Acceptance criteria

- The test user sees the tile and the app opens with data.

Story P2.4 - Cache toolbox

Goal: Know which cache to clear when changes do not show.

Your tasks

1. List the caches: browser, UI5 resource/Component preload, launchpad client cache, launchpad global cache, gateway metadata and model caches, UI5 application index.
2. Find the transactions/reports that clear each one and the order in which you use them.
3. Make a change (for example a tile title in the catalog), observe the delay, then clear caches until it shows.

Where to look / topics to explore

- /UI2/ cache invalidation reports, gateway cache cleanup (frontend and backend), UI5 app index calculation

Acceptance criteria

- A cheat sheet exists: symptom -> cache -> tool.

Story P2.5 - Debug toolbox

Goal: Find errors in Gateway, UI5 and ABAP.

Your tasks

1. Provoke three errors: remove a role (no tile / 403), deactivate the service (404), cause an exception in a data read (500).
2. For each find the cause using: gateway error logs (frontend and backend), runtime errors, application log, browser network tab and console.
3. Debug UI5: load unminified sources, disable view cache, open technical information dialog, inspect the UI5 control tree.
4. Debug ABAP: set an external breakpoint and trigger it from the app.

Where to look / topics to explore

- sap-ui-debug URL parameter, UI5 Diagnostics (keyboard shortcut), /IWFND/ERROR_LOG, /IWBEP/ERROR_LOG, ST22, SLG1

Acceptance criteria

- You fixed all three errors and wrote the diagnosis path.

Milestone checkpoint

Self-test questions (write your own answers)

1. Which two things must exist for a standard app to run?
2. Tile not updated after change: first suspect?

Project P3 - Extend a Standard List Report

Item	Detail
Mini project	Extend one standard (or self-built) Fiori elements list report without touching the original code: hide a field, add a column, add a filter, add a button with logic.
Level / effort	Intermediate to Advanced / 6-8 hours
Topics practised	Key user adaptation, developer adaptation project, extension points, controller extension, upgrade safety, transport.

Learning outcomes
- Know all extension levels and choose the right one.
- Build and deploy an adaptation project.

Story P3.1 - Choose base app and map the extension options

Goal: Know what you may change.

Your tasks

1. Choose a base: a standard list report on your S/4 system (preferred) or your own Fiori elements V2 list report built on the mock backend.
2. Write a table: extension level (key-user adaptation, adaptation project, extension points, modification/copy), who does it, tool, upgrade safety.
3. List the extension points of your app (view, controller, fragments) from its manifest and documentation.

Where to look / topics to explore
- SAP Fiori tools: Adaptation Project
- Extension points of Fiori elements V2/V4 list reports

Acceptance criteria
- Your table and the extension point list exist.

Story P3.2 - Key-user adaptation

Goal: No-code changes.

Your tasks

1. Open the app in adaptation mode in the launchpad.
2. Hide a field, rename a column label, move a column, add a custom field if your system supports it.
3. Save, publish and test as another user; then reset.

Where to look / topics to explore
- Adapt UI (key user), flexibility changes

Acceptance criteria
- Changes visible to the test user; reset works.

Story P3.3 - Adaptation project with code

Goal: A real developer extension.

Your tasks

1. Create an adaptation project for the base app with the Fiori tools generator.
2. Add a custom column with a fragment showing a computed value or status colour.

3. Add a custom filter or a new selection field if the app type allows it.
4. Add a toolbar button with a controller extension that opens a dialog or calls a function import/action.
5. Test in preview against the real system.

Where to look / topics to explore
- Adaptation project: add fragment, add controller extension, change page configuration

Acceptance criteria
- Column, filter and button are visible and work.

Story P3.4 - Deploy, transport, upgrade check

Goal: Make it safe to keep.

Your tasks

1. Deploy the adaptation project to the system (package and transport).
2. Explain how SAP upgrades affect your extension and what you test after an upgrade.
3. Write a short extension document: what, why, extension type, owner.

Acceptance criteria
- Project is deployed; upgrade checklist written.

Milestone checkpoint

Self-test questions (write your own answers)
1. Why prefer an adaptation project to copying the standard app?

Project P4 - Deploy a UI5 App to BTP and to S/4HANA

Item	Detail
Mini project	Deploy the freestyle CRUD app from P1 to SAP BTP (Cloud Foundry) and to an S/4 ABAP repository and open it from a launchpad in both worlds.
Level / effort	Intermediate / 6-8 hours
Topics practised	HTML5 application repository, managed approuter or Work Zone, MTA build, ABAP repository deployment, transports, launchpad registration.

Learning outcomes

- Deploy the same app to both targets.
- Know the differences in routing, authentication and launchpad setup.

Story P4.1 - Prepare the app for deployment

Goal: A clean build.

Your tasks

1. Make the app id, namespace, title and i18n final; remove test data and debug code.
2. Build with the UI5 tooling to create the optimised dist (Component-preload).
3. Make the OData service URL configurable (BTP destination versus S/4 relative path).

Where to look / topics to explore

- ui5 build, manifest dataSources, destinations

Acceptance criteria

- Build output contains the preload file; manifest has no hard-coded host.

Story P4.2 - Deploy to SAP BTP

Goal: App served from HTML5 repo behind an approuter.

Your tasks

1. Add the Cloud Foundry deployment configuration with the Fiori tools generator (HTML5 repository and destination, managed approuter or standalone).
2. Create the destination to your CAP backend in the BTP cockpit.
3. Build the MTA archive and deploy it.
4. Open the app through the HTML5 apps URL or SAP Build Work Zone: create a content provider, add the app, tile and role, and open it from the site.

Where to look / topics to explore

- SAP Fiori tools: add deployment configuration (Cloud Foundry), mbt build, cf deploy, Work Zone content manager

Acceptance criteria

- The app opens from Work Zone and shows data via the destination.

Story P4.3 - Deploy to S/4HANA (ABAP repository)

Goal: App stored in the ABAP system.

Your tasks

1. Add ABAP deployment configuration (target system, package, BSP application name, transport).
2. Deploy from the command line and check the BSP application in the repository.

3. Create launchpad content: semantic object, target mapping, tile, catalog, role; assign to a user.
4. Open it from the launchpad as the user.

Where to look / topics to explore
- fiori deploy / ui5-task-deploy-to-abap
- BSP application naming (Z/Y), launchpad designer

Acceptance criteria
- Tile in the S/4 launchpad opens the deployed app with data.

Story P4.4 - Compare and troubleshoot

Goal: Understand both targets.

Your tasks

1. Write a comparison: hosting, auth (XSUAA/IAS versus ABAP), routing (destination versus ICF/relative), cache, versioning, rollback.
2. Break each deployment on purpose (wrong destination; missing catalog) and fix it.

Acceptance criteria
- Comparison and fixes documented.

Milestone checkpoint

Self-test questions (write your own answers)
1. Where does the browser get the OData data from after deployment on BTP?
2. Which object makes the app visible in the S/4 launchpad?

Project P5 - UI5 Showroom: Navigation, Control, Filter, Group, Currency, Time Offset, CSS, Authorization

Item	Detail
Mini project	One small app "Order Showroom" (two tiny apps for navigation) that demonstrates every item of your list on the mock backend.
Level / effort	Intermediate / 10-14 hours
Topics practised	Cross-application navigation, custom control, filter and group, currency formatting, time zones, custom CSS, role-based visibility simulation.

Learning outcomes
- Practise eight typical UI5 interview and project topics in one place.

Story P5.1 - Cross-app navigation

Goal: Click an order, open the customer app with parameters.

Your tasks

1. Create two small apps: Orders (list) and Customers (detail). Run both in a local launchpad sandbox with the intents Order-display and Customer-display.
2. In the Orders list add a link on the customer name that navigates to Customer-display with the customer id parameter.
3. In the customer app read the startup parameter and bind the page to that customer.
4. Add a back navigation and test with the browser back button.

Where to look / topics to explore
- sap.ushell Navigation service (or CrossApplicationNavigation), sandbox config, startupParameters

Acceptance criteria
- Navigation works and the customer opens with the right data.

Story P5.2 - Custom control

Goal: A reusable StatusBadge control.

Your tasks

1. Create control StatusBadge with properties text and state (Success, Warning, Error), a press event and its renderer (API version 2).
2. Use it in the order table instead of plain text for status.
3. Add a theme-friendly CSS class and keyboard accessibility (focusable, Enter/Space triggers press).

Where to look / topics to explore
- sap.ui.core.Control.extend, renderer apiVersion 2, rm.openStart/class/openEnd

Acceptance criteria
- Badge shows colours by state and fires press; works with keyboard.

Story P5.3 - Filter and group

Goal: Search, filter and group the order list.

Your tasks

1. Add a SearchField (live search on order number and customer) and a category dropdown filter combined with AND.
2. Add a Group by switch: category or status, with group headers showing the count and sum of amounts.
3. Keep filter and group state when navigating away and back.

Where to look / topics to explore

- Filter, FilterOperator, Sorter with group, groupHeaderFactory, binding.filter

Acceptance criteria

- Filter + group work together; group headers show count and total.

Story P5.4 - Currency

Goal: Correct amounts per currency.

Your tasks

1. Display amount with currency using the proper type so EUR, USD and INR show the correct decimals and symbol/code.
2. Show a total per currency (never add different currencies).
3. Switch the user locale (de, en-US, en-IN) and observe the number format.

Where to look / topics to explore

- sap.ui.model.type.Currency, NumberFormat.getCurrencyInstance, locale configuration

Acceptance criteria

- Amounts are formatted by currency and locale.

Story P5.5 - Time offset

Goal: Timestamps correct for any user timezone.

Your tasks

1. Backend delivers UTC timestamps. Show orderedAt in the browser timezone.
2. Add a timezone selector (for example Asia/Kolkata, Europe/Berlin, America/New_York) and show the time and the UTC offset for the selected zone.
3. Handle a date-only field (no timezone shift) versus a timestamp and write why they behave differently.
4. Test around a daylight saving change.

Where to look / topics to explore

- DateFormat.getDateTimeWithTimezoneInstance, Timezone, UI5 configuration timezone, Intl.DateTimeFormat

Acceptance criteria

- The same timestamp shows different local times and offsets per selected zone.

Story P5.6 - CSS

Goal: Custom styling the right way.

Your tasks

1. Add a custom stylesheet via the manifest and a style class on key controls (addStyleClass).
2. Style the badge and table rows (zebra, hover) using theme parameters where possible; avoid !important.
3. Add one media-query rule for phone and one for desktop; test with device emulation.

4. Check the app in two themes (Horizon and Quartz) and fix contrast problems.

Where to look / topics to explore
- sap.ui5/resources/css, Parameters API, sapUiResponsiveMargin

Acceptance criteria
- Styling works in both themes and on phone width.

Story P5.7 - Authorization simulation

Goal: Show or enable a button only when the user has the role.

Your tasks

1. Create a user model with id and roles. Fill it from the backend function me() and, for testing, from a URL parameter like ?role=ADMIN.
2. Show the Delete button only for ADMIN, enable Approve only for MANAGER or ADMIN, show a read-only message for others.
3. Move the check to one reusable helper (formatter or model function), not scattered conditions.
4. Add the backend rule too: delete requires the role on the server. Test with a user without the role and note the difference between hiding and securing.

Where to look / topics to explore
- Expression binding visible="{= ... }", JSONModel, formatters, CAP @restrict

Acceptance criteria
- Buttons react to the role; direct API call without role is refused by the backend.

Milestone checkpoint

Self-test questions (write your own answers)
1. Why is hiding a button not security?
2. Why not add amounts of different currencies?